

Creating Images–Processing

To create images with programs, we can use processing. What we do in processing is basically defining functions; id est, teaching the computer what to do as a response to certain events (be it the player pressing a button, the player clicking the mouse, or simply the time passing).	
Code Format (Defining Functions)	Execution (Running Functions)
<pre>def setup(): # Mandatory. size(width, height) # First function in setup(). (codeblock) def draw(): # Mandatory. (codeblock) # Statement or return None. def mousePressed(): # Optional. (codeblock) def mouseReleased(): # Optional. (codeblock) def ... # Optional.</pre>	First, function setup() will be runned one time. Then, function draw() will be runned at each time interval. Id Est: setup() draw() draw() draw() draw() draw() ... (Optional) Whenever mouse is pressed, function mousePressed() will be runned; whenever mouse is released, function mouseReleased() will be runned; ... (so on and so forth). ※ Images will be shown in the window. ※ Other outputs will be printed in the console (the dark rectangle at the bottom of the processing environment).
# The setup() function tells the program what to do at the start and the draw() function tells the program what to do after the start; they are necessary to make the program functional and therefore should be defined. # The setup() function should contain the size(width, height) function as the first appearing function. # The draw() function should be defined, but it can be “return None” which means doing nothing after the start.	

Global Statement	
Variables are assumed to be used and assigned inside functions. To make a variable available outside the function or to access variables from somewhere outside the function, global the variable.	global x, y, z

Photo and Coordinates

Colour		Opacity
RGB System (r, g, b) ※ The numbers represent how much red, green, and blue are to be mixed together. ※ Domain: r, g, b, ∈ [0, 255]	Hexadecimal System (“#Hex”) ※ Each colour has a hex label. ※ Quotations and pound keys. ※ The hex has exactly 6 digits.	How visible an object is (0 ~ 255). Opacity = 0 → Totally Transparent Opacity = 255 → Totally Visible Default 255. (Opacity is how “visible” an object is, whereas transparency is how “transparent” an object is. Normally, opacity has a range of 0%~100%, but not in processing.)
Conversions The hex label of a colour can be determined with the r, g, and b values by converting ($r \times 16^4 + g \times 16^2 + b \times 16^0$) into hexadecimal form. E.G. Colour “#FF0100” would be $r = FF_{16} = 255_{10}$ and $g = 1, b = 0$.		
Other Colour Systems Such as HSB.		

Coordinate System in Processing	
Points	(X, Y) refers to the pixel X units and Y units from the left and from the up, with the most upper-left pixel being (0, 0) the origin. X and Y can also be outside of the screen, such as (-5, -5) or (width+5, height+5).
Polars	Directed angle fromV(1,0) to V(0,1) is (+0.5π), meaning positive angles are going in clockwise direction.

Sketching

Sketching Properties		
size	size(width, height)	※Set up size of window (units in pixels).
colour mode	colorMode(mode, a, b, c, d) colorMode(mode, a)	※The ranges of R, G, B, (or H, S, B,) and opacity are 255 by default but can be changed by the colorMode() function. ※Remember to state which colour system—mode—are you changing ranges.
colour	haha = color(...)	※Store information of a specific colour in variable haha.
background colour	background(...)	1 Parameter (a): RGB=(a, a, a) or “a” as a hex code or as a colour variable. 2 Parameters (a, alpha): RGB = (a, a, a) and opacity = alpha. 3 Parameters (r, g, b): RGB = (r, g, b). 4 Parameters (r, g, b, alpha): RGB = (r, g, b) and opacity = alpha.
stroke	stroke(...)	※Determine stroke colour. Parameters same as background().
	noStroke()	※No strokes while sketching.
	strokeWeight(weight)	※How thick (unit in pixels) the stroke should be.
fill	fill(...)	※Determine fill colour. Parameters same as background().
	noFill()	※No fill while sketching.

Sketching Points, Lines, and Surfaces		
When drawing a shape, the fill and stroke would be determined by the most recent fill() and stroke() given. For points and lines, the colour depends on the most recent stroke() given.		
point	point(x, y)	※Draw point at coordinate (x, y).
line	line(x_1, y_1, x_2, y_2)	※Draw line with endpoints (x_1, y_1) and (x_2, y_2).
ellipse	ellipse(x, y, width, height)	※Draw stretched circle with centre (x, y) and the width, and the height.
rectangle	rect(x, y, width, height)	※Draw rectangle the width, the height and upper-left vertex being (x, y).
arc	arc(x, y, w, h, s, e)	※Draw an arc, a part of an ellipse with centre (x, y), width w, and height h. ※The start and end angles are specified with variables s and e in radians (think about the start and end as directed angles on polar coordinates).

Sketching Texts		
Text colour depends on the most recent fill() given.		
text size	textSize(num)	※Set the size of text (unit in pixels). Size of text refers to height of shown characters.
text align	textAlign(option)	※Options are LEFT, RIGHT, or CENTER.
text	text(String, a, b)	※Align the characters at $x = a$ and let them stand on horizon $y = b$.
※Functions print() and println()—PRINTLN()—which is the same in python mode, will show the output in the console.		

Images

**The properties of a pasted image is determined by the most recent tint() function given.
Images should be loaded in setup().**

load image	haha = loadImage(“pict.png”)	※Load image file (they can be .png .jpg .gif .tga) from sketch folder. ※We would refer to the image as variable “haha” from now on. ※Only .png files allow transparent background.
tint	tint(...)	※Change overall colour style of image, with parameters similar to fill().
image	image(haha, x, y)	※Display haha (image) with its most upper-left pixel at coordinate (x, y).

Frames

Processing runs draw() function repeatedly, each time creating a new frame. Multiple frames create animations.		
count	frameCount	The counting of frames. (E.G. In the second frame, frameCount would return 2.)
rate	frameRate(num)	Set frame rate, which is the number of frames each second, default 60. Only when frame rate between 30 ~ 60 will human eyes recognize it as animations.
save	saveFrame(“pict.png”)	Save present frame is image file (they can be .png .jpg .gif .tga) in sketching folder.

Loops

Processing runs draw() function repeatedly by default, creating a loop. However, you can stop it with functions.		
stop	noLoop()	Stop draw() from running repeatedly. (No parameters.)
resume	loop()	Continue running draw() repeatedly after stopping it at some point.
run once	redraw()	When an event occurs, run draw() function 1 time (instead of repeatedly).

Events

Mouse (Press ~ ~ ~ ~) + (Release) = (Click)		
coordinates	mouseX & mouseY pmouseX & pmouseY	System variables that return coordinates of cursor (relative to processing window). Variables pmouseX and pmouseY return previous coordinates.
mouse button	mouseButton	Return pressed part (mouseButton == LEFT or RIGHT or CENTER).
mouse pressed	mousePressed	Boolean, only True if mouse pressed throughout the entire frame.
	def mousePressed():	A function that runs when the mouse is pressed.
mouse released	def mouseReleased():	A function that runs when the mouse is released.
mouse clicked	def mouseClicked():	A function that runs when the mouse is pressed and then released.
mouse moved	def mouseMoved():	A function that runs when the mouse is moving.
mouse dragged	def mouseDraged():	A function that runs when the mouse is moving and also pressed.

Keyboard

Keys can be designed to have features instead of representing alphabets. Keys with those features are “coded.” Such as (keyCode == UP or DOWN or LEFT or RIGHT). Note that not all keys are coded.

key pressed	def keyPressed():	A function that runs when any key is pressed.
key	key	System variable that returns an alphabet with single quotes or key status. E.G. (key == 'A' #must be single quote) or (ord(key) == 65) or (key == CODED)
key code	keyCode	Returns the feature the pressed key is designed to have.
key released	def keyReleased():	A function that runs when any key is released.

Cursor

The cursor can be controlled—when it goes inside the output window—with cursor functions.		
disappear	noCursor()	Function noCursor() hides the cursor when it gets inside the output window.
appear	cursor()	0 Argument: Show cursor (after hiding it). 1 Argument: Change shape of cursor. E.G. cursor(ARROW) or cursor(TEXT) or cursor(HAND) or cursor(WAIT) or cursor(MOVE) or cursor(CROSS)

	default
	crosshair
	hand
	move
	text
	wait

Audio

After importing the “Minim” library (click “Sketch,” “Import Library,” “Add Library,” search for “Minim,” select and install), you can deal with audio in processing.

<pre>add_library(“Minim”) # or “minim” haha = Minim(this) def ... (codeblock) global haha hehe = haha.loadFile(“songname.mp3”) hehe.play() # Other Functions # hehe.pause() # hehe.rewind() # hehe.loop()</pre>	library	Add library to code.
	object	Create object to deal with audio files in “this” piece of code. (Just as you cannot append elements to scalar variables, you cannot directly assign audio files to variables.) The object you create should also be global-ed.
	code	Other portions of code.
	load	Load audio file using the object created.
	play	Play audio. (Or you can also pause, rewind, or loop the audio.) Note that the audio would be discarded after the play() function. To play it again, you need to load it again.